

LAPORAN PRAKTIKUM
STRUKTUR DATA
IMPLEMENTASI *BINARY TREE* DAN *GRAPH TRAVERSAL*
MENGGUNAKAN BAHASA *JAVA*



Disusun Oleh:

Haliya Isma Husna Putri Ahmadi
2511532002

Dosen Pengampu:
Wahyudi. Dr. S.T.M.T

Asisten Praktikum:
Muhammad Galid

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan Syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas Rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum Struktur Data pekan dengan judul “Implementasi *Binary Tree* dan *Graph Traversal* Menggunakan Bahasa *Java*” tepat pada waktunya.

Penyusunan laporan ini bertujuan untuk memenuhi tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran bagi penulis dalam memahami konsep memahami konsep struktur data pohon (*tree*) dan graf (*graph*) beserta teknik penelusurannya dalam pemrograman *Java*. Melalui praktikum ini, penulis mempelajari implementasi *Binary Tree*, proses pembentukan simpul (*node*), serta berbagai metode *traversal* seperti *Inorder*, *Preorder*, dan *Postorder*. Selain itu, penulis juga mempelajari representasi graf menggunakan *adjacency list* serta algoritma penelusuran graf *Depth First Search (DFS)* dan *Breadth First Search (BFS)*.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Ucapan terima kasih yang sebesar-besarnya penulis sampaikan kepada dosen pengampu, asisten praktikum, serta semua pihak yang terlibat dalam pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca.

Padang, 10 June 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB I PENDAHULUAN.....	5
1.1 Latar Belakang.....	5
1.2 Tujuan	6
1.3 Manfaat	6
BAB II PEMBAHASAN.....	7
2.1 Dasar Teori.....	7
2.1.1 <i>Binary Tree</i>	7
2.1.2 <i>Graph</i>	9
2.2 Langkah Kerja	11
2.2.1 Pembuatan <i>Package</i> dan <i>Class</i>	11
2.2.2 Membuat Program <i>Node_2511532002</i>	15
2.2.3 Membuat Program <i>Btree_2511532002</i>	18
2.2.4 Membuat Program <i>BtreeDriver_2511532002</i> {.....	22
2.2.5 Membuat Program <i>GraphTravesal_2511532002</i>	26
2.2.6 <i>Upload</i> ke GitHub	31
2.3 Hasil.....	33
2.3.1 Hasil <i>Node_2511532002</i>	33
2.3.2 Hasil <i>Btree_2511532002</i>	34
2.3.3 Hasil <i>BtreeDriver_2511532002</i>	34
2.3.4 Hasil <i>GraphTraversal_2511532002</i>	34
BAB III KESIMPULAN.....	35
3.1 Kesimpulan.....	35
3.2 Saran	35
DAFTAR PUSTAKA.....	36

TUGAS PRAKTIKUM PEKAN 5.....	37
1. Kode Program	37
1.1 Graph_2511532002	37
1.2 Kelas <i>GraphPanel</i>_2511532002.....	40
2. Hasil.....	50
3. Kesimpulan	51

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan Struktur data merupakan salah satu konsep fundamental dalam ilmu komputer untuk mengorganisasi, menyimpan, dan mengelola data agar dapat diakses serta diproses secara efisien. Pemilihan struktur data yang tepat sangat berpengaruh terhadap kinerja suatu program. Oleh karena itu, pemahaman mengenai berbagai jenis struktur data menjadi hal yang penting dalam mengembangkan perangkat lunak yang efektif dan efisien.

Salah satu struktur data yang banyak digunakan adalah *Binary Tree*, yaitu struktur data hierarkis. *Binary Tree* banyak dimanfaatkan dalam berbagai aplikasi komputer seperti representasi data hierarkis, sistem basis data, kompilator, dan algoritma pencarian. Dalam penggunaannya, *Binary Tree* dapat ditelusuri menggunakan beberapa metode *traversal*, yaitu *Preorder Traversal*, *Inorder Traversal*, dan *Postorder Traversal*, yang masing-masing memiliki urutan kunjungan simpul yang berbeda sesuai kebutuhan pemrosesan data.

Selain struktur data pohon, struktur data *Graph* juga memiliki peranan penting dalam ilmu komputer. *Graph* digunakan untuk merepresentasikan hubungan antar objek melalui simpul (*vertex*) dan sisi (*edge*). Struktur ini banyak diterapkan pada jaringan komputer, media sosial, sistem navigasi, dan berbagai permasalahan optimasi lainnya. Untuk menelusuri suatu graph, digunakan algoritma *traversal* seperti *Depth First Search (DFS)* dan *Breadth First Search (BFS)* yang memiliki karakteristik dan pendekatan berbeda dalam mengunjungi setiap simpul.

Melalui praktikum ini, penulis diharapkan dapat memahami konsep implementasi *Binary Tree* dan *Graph* menggunakan bahasa pemrograman *Java*. Selain membangun struktur datanya, praktikum ini juga mengimplementasikan berbagai metode *traversal* untuk memahami cara kerja penelusuran data pada kedua struktur tersebut.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

- 1.2.1 Memahami konsep dasar ata *Binary Tree* dan *Graph*.
- 1.2.2 Mengimplementasikan *Binary Tree* serta operasi *traversal* seperti *Inorder*, *Preorder*, dan *Postorder* menggunakan bahasa pemrograman *Java*.
- 1.2.3 Memahami representasi *Graph* menggunakan *adjacency list*, dengan algoritma *Depth First Search (DFS)* dan *Breadth First Search (BFS)*.
- 1.2.4 Melatih kemampuan pemrograman dan pemahaman struktur data dalam *Java*.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum ini antara lain:

- 1.3.1 Menambah pemahaman mengenai konsep dan karakteristik *Binary Tree* serta *Graph*.
- 1.3.2 Memahami cara kerja berbagai metode *traversal* baik pada *Binary Tree* atau mekanisme penelusuran pada *Graph*.
- 1.3.3 Melatih kemampuan logika pemrograman dalam mengelola dan mengurutkan data secara sistematis.

BAB II

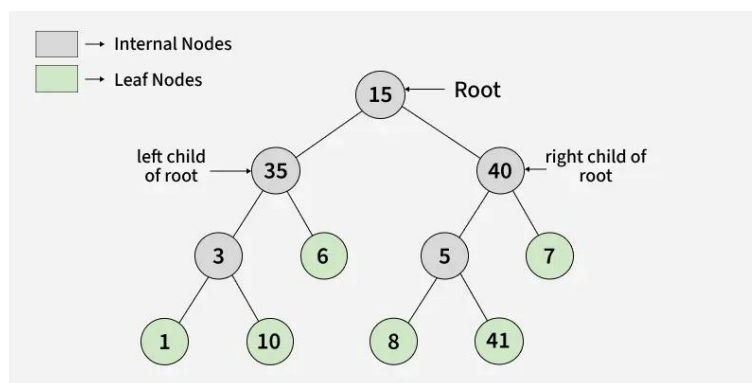
PEMBAHASAN

2.1 Dasar Teori

Struktur data merupakan cara untuk mengorganisasi dan menyimpan data dalam komputer sehingga data dapat diakses, dikelola, dan diproses secara efisien. Pemilihan struktur data yang tepat sangat penting karena dapat memengaruhi kinerja algoritma dan efisiensi penggunaan memori. Dalam ilmu komputer, terdapat berbagai jenis struktur data yang digunakan sesuai kebutuhan, salah satunya adalah struktur data *non-linear* seperti *Tree* dan *Graph*.

Tree merupakan struktur data hierarkis yang terdiri dari kumpulan simpul (*node*) yang saling terhubung melalui hubungan *parent* dan *child*. Salah satu bentuk *tree* yang paling umum digunakan adalah *Binary Tree*, yaitu pohon yang setiap simpulnya memiliki maksimal dua anak. Sementara itu, *Graph* merupakan struktur data yang digunakan untuk merepresentasikan hubungan antar objek melalui simpul (*vertex*) dan sisi (*edge*). *Graph* banyak digunakan dalam berbagai aplikasi seperti jaringan komputer, sistem navigasi, media sosial, dan pemetaan hubungan data.

2.1.1 *Binary Tree*



Binary Tree adalah struktur data pohon yang setiap simpulnya memiliki paling banyak dua anak, yaitu anak kiri (*left child*) dan anak kanan (*right child*). Simpul paling atas pada pohon disebut *root*, sedangkan simpul yang tidak memiliki anak disebut *leaf node*.

Binary Tree digunakan untuk menyimpan data secara hierarkis dan memungkinkan proses pencarian serta pengelolaan data dilakukan dengan lebih terstruktur.

Karakteristik utama *Binary Tree* meliputi:

1. Memiliki *satu root node*.
2. Setiap *node* dapat memiliki maksimal dua *child*.
3. Setiap *child* dapat menjadi *parent* bagi *node* lainnya.
4. Tidak memiliki siklus (*cycle*).

Traversal pada *Binary Tree*

Traversal adalah proses mengunjungi seluruh simpul dalam suatu pohon dengan urutan tertentu. *Traversal* digunakan untuk membaca, menampilkan, atau memproses data yang tersimpan pada *Binary Tree*.

1. *Preorder Traversal*

Preorder Traversal mengunjungi simpul dengan urutan:

Root → *Left Subtree* → *Right Subtree*

Metode ini mengunjungi *root* terlebih dahulu, kemudian menelusuri *subtree* kiri dan *subtree* kanan secara rekursif. *Traversal* ini sering digunakan untuk menyalin atau merepresentasikan struktur pohon.

2. *Inorder Traversal*

Inorder Traversal mengunjungi simpul dengan urutan:

Left Subtree → *Root* → *Right Subtree*

Metode ini menelusuri *subtree* kiri terlebih dahulu, kemudian *root*, dan terakhir *subtree* kanan. Pada *Binary Search Tree (BST)*, *traversal* ini menghasilkan data yang terurut secara *ascending*.

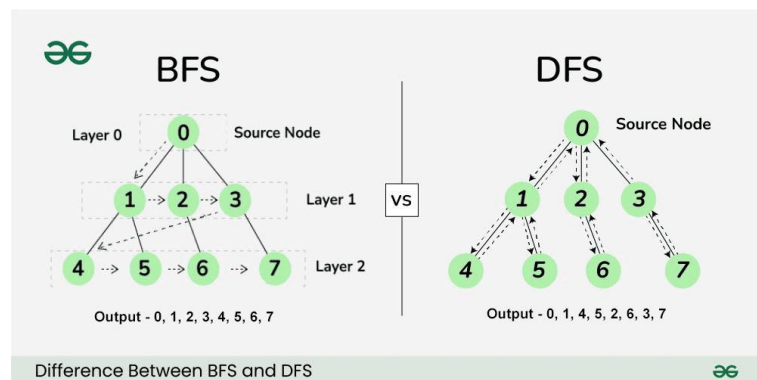
3. Postorder Traversal

Postorder Traversal mengunjungi simpul dengan urutan:

Left Subtree $\rightarrow$ *Right Subtree* $\rightarrow$ *Root*

Metode ini mengunjungi root terakhir setelah seluruh *subtree* kiri dan kanan selesai diproses. *Traversal* ini sering digunakan dalam proses penghapusan pohon dan evaluasi ekspresi.

2.1.2 Graph



Graph merupakan struktur data yang terdiri dari sekumpulan simpul (*vertex*) dan hubungan antar simpul yang disebut *edge*. Berbeda dengan *Tree* yang bersifat hierarkis, *Graph* memiliki struktur yang lebih fleksibel karena satu simpul dapat terhubung dengan banyak simpul lainnya.

Graph dapat direpresentasikan dalam berbagai bentuk, salah satunya menggunakan *Adjacency List*. Representasi ini menyimpan daftar tetangga dari setiap simpul sehingga penggunaan memori menjadi lebih efisien terutama pada *graph* yang memiliki sedikit *edge*.

Komponen utama *Graph* terdiri dari:

1. *Vertex* (simpul).
2. *Edge* (sisi atau hubungan).
3. *Path* (jalur).
4. *Degree* (jumlah hubungan pada suatu simpul).

a) *Depth First Search (DFS)*

Depth First Search (DFS) merupakan algoritma *traversal graph* yang bekerja dengan menelusuri simpul sedalam mungkin sebelum kembali ke simpul sebelumnya. *DFS* menggunakan pendekatan rekursif atau struktur data stack untuk menyimpan simpul yang akan dikunjungi.

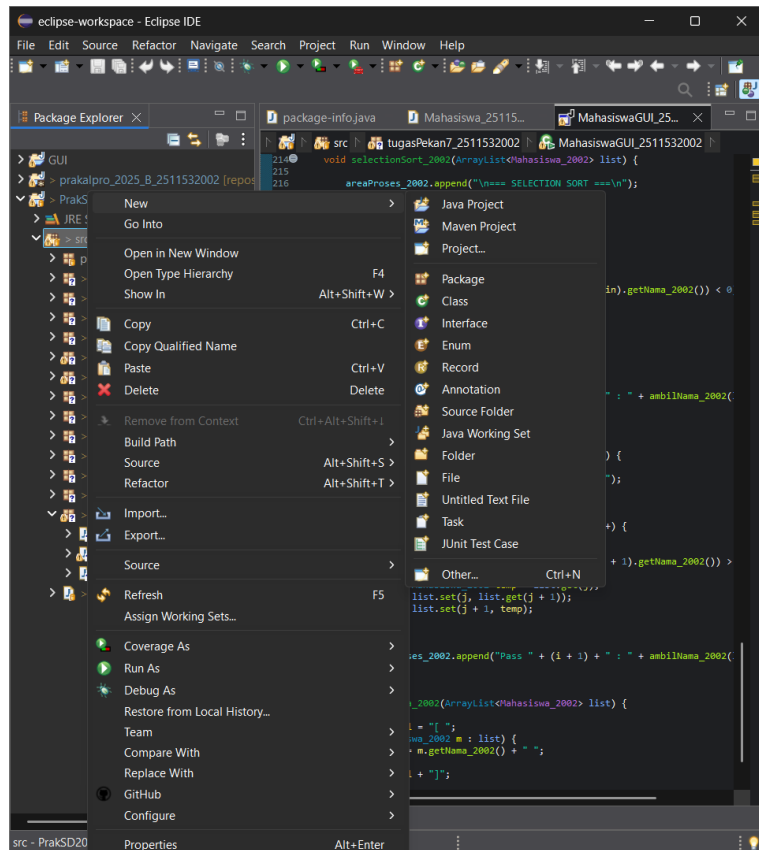
b) *Breadth First Search (BFS)*

Breadth First Search (BFS) merupakan algoritma *traversal graph* yang menelusuri simpul berdasarkan tingkat kedekatannya dengan simpul awal. *BFS* menggunakan struktur data queue sehingga simpul yang pertama masuk akan diproses terlebih dahulu.

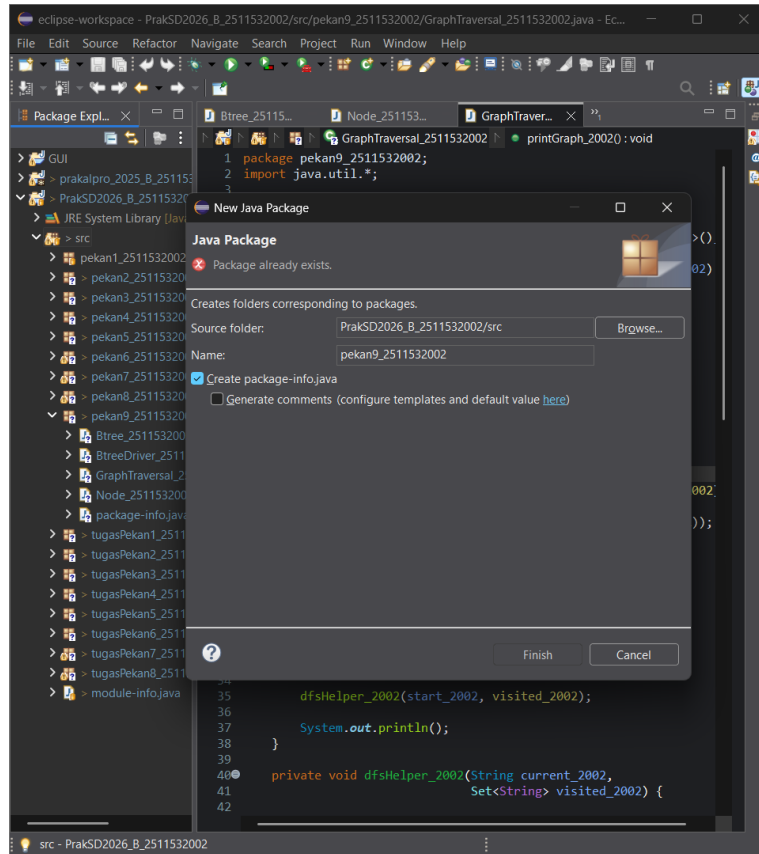
2.2 Langkah Kerja

2.2.1 Pembuatan *Package* dan *Class*

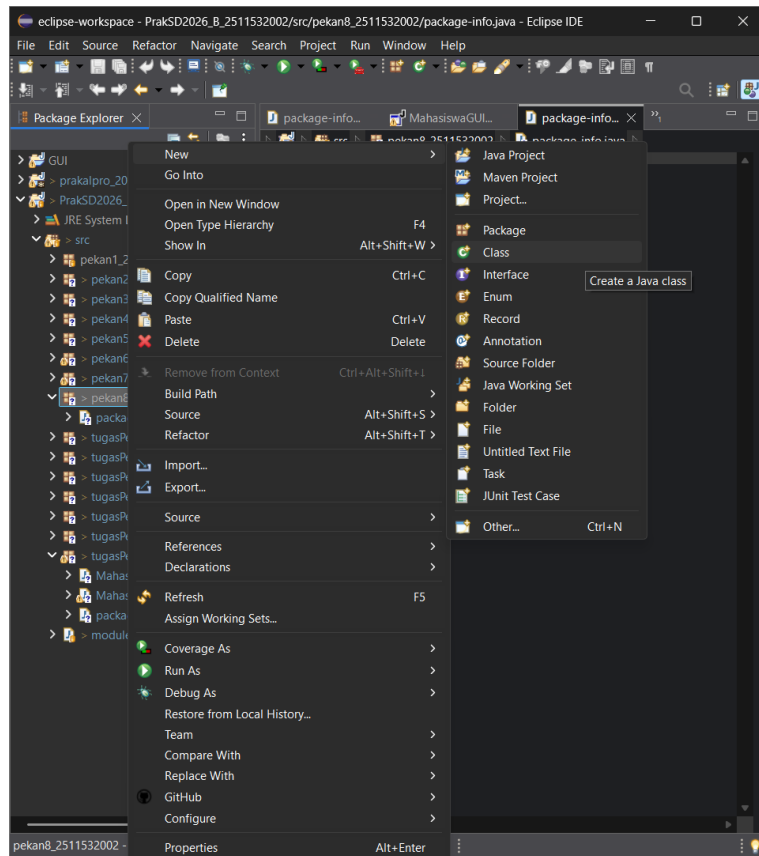
- 1) Sebelum membuat suatu program *user* perlu untuk membuat *package* terlebih dahulu. Klik kanan pada *src* pada *repository* “PrakSD2026_B_2511532002” lalu pilih “New” dan klik “Package”.



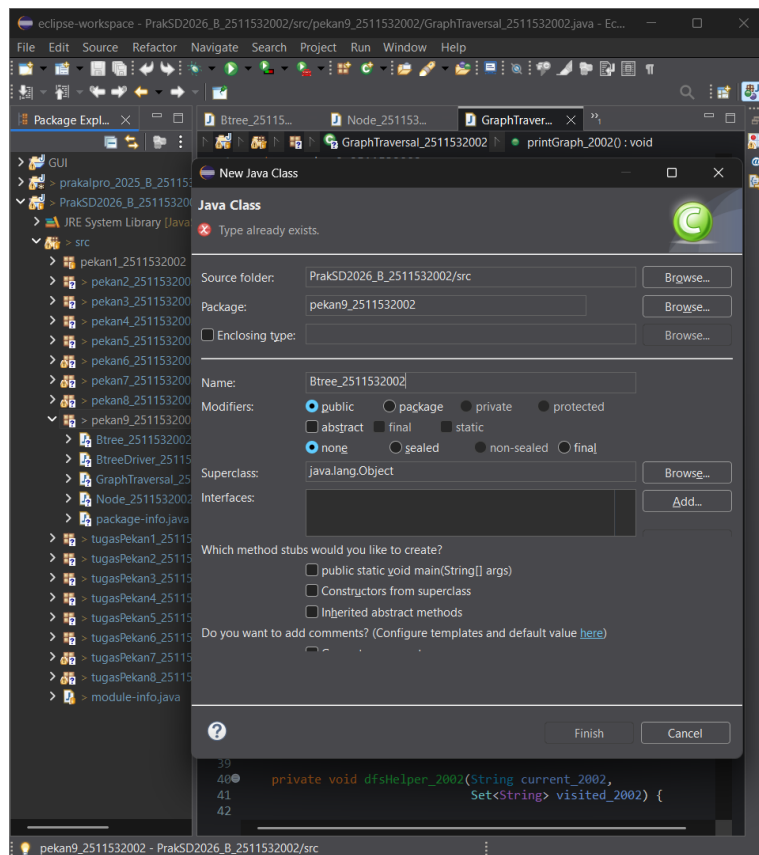
- 2) Setelah itu akan muncul *pop up* “Java Package”, buat nama *package* dengan ketentuan tanpa *capslock*, *space*, dan karakter khusus lainnya. Lalu klik “*finish*”.



- 3) Selanjutnya adalah membuat membuat *class*. pada package yang telah dibuat tapi klik kanan lalu pilih “New” dan klik “Class”.



- 4) Setelah itu pada menu “*Java Class*” buat nama *class*, dengan ketentuan nama harus capslock diawal kalimat dan tanpa spasi.



2.2.2 Membuat Program *Node_2511532002*

```

package pekan9_2511532002;

public class Node_2511532002 {
    int data_2002;
    Node_2511532002 left_2002;
    Node_2511532002 right_2002;
    public Node_2511532002 (int data_2002) {
        this.data_2002=data_2002;
        left_2002=null;
        right_2002=null;
    }
    public void setLeft_2002 (Node_2511532002 node_2002) {
        if (left_2002 == null)
            left_2002 = node_2002;
    }
    public void setRight_2002 (Node_2511532002 node_2002) {
        if (right_2002 == null)
            right_2002= node_2002;
    }
    public Node_2511532002 getLeft() {
        return left_2002;
    }
    public Node_2511532002 getRight_2002() {
        return right_2002;
    }
    public int getData_2002() {
        return data_2002;
    }
    public void setData_2002 (int data_2002) {
        this.data_2002=data_2002;
    }
    void printPreorder_2002 (Node_2511532002 node_2002) {
        if (node_2002 == null)
            return;
        System.out.print(node_2002.data_2002+" ");
        printPreorder_2002(node_2002.left_2002);
        printPreorder_2002(node_2002.right_2002);
    }
    void printPostorder_2002 (Node_2511532002 node_2002) {
        if (node_2002 == null)
            return;
        printPostorder_2002(node_2002.left_2002);
        printPostorder_2002(node_2002.right_2002);
        System.out.print(node_2002.data_2002+" ");
    }
    void printInorder_2002(Node_2511532002 node_2002) {
        if (node_2002==null)
            return;
        printInorder_2002(node_2002.left_2002);
        System.out.print(node_2002.data_2002+" ");
        printInorder_2002(node_2002.right_2002);
    }
    public String print() {
        return this.print("", true,"");
    }
    public String print(String prefix, boolean isTail, String sb) {
        if (right_2002!=null) {
            right_2002.print(prefix + (isTail ? "| " : " "), false, sb);
        }
        System.out.println(prefix+(isTail ? "\\--":"/--")+data_2002);
        if (left_2002 != null) {
            left_2002.print(prefix+(isTail? " ":"| "), true, sb);
        }
        return sb;
    }
}

```

Setelah membuat *class Node_2511532002*, maka mahasiswa dapat menulis *syntax* nya.

1. Membuat *class Shell Sort*

```
public class Node_2511532002 {
```

Syntax di atas digunakan untuk membuat class bernama *Node_2511532002* sebagai tempat penulisan program algoritma *Shell Sort*.

2. Membuat *Constructor Node*

```
public Node_2511532002 (int data_2002) {
    this.data_2002=data_2002;
    left_2002=null;
    right_2002=null;
}
```

Constructor Node_2511532002() digunakan untuk membuat objek *node* baru. *Constructor* ini menerima sebuah nilai data yang akan disimpan pada *node*, kemudian menginisialisasi *child* kiri dan *child* kanan dengan nilai null karena *node* yang baru dibuat belum memiliki anak.

3. Membuat *Method Setter dan Getter*

```
public void setLeft_2002 (Node_2511532002 node_2002)
public void setRight_2002 (Node_2511532002 node_2002)
public Node_2511532002 getLeft()
public Node_2511532002 getRight_2002()
public int getData_2002()
public void setData_2002 (int data_2002)
```

Method setter dan *getter* digunakan untuk mengatur serta mengambil nilai yang terdapat pada *node*. *Method setLeft_2002()* dan *setRight_2002()* digunakan untuk menghubungkan *node* dengan *child* kiri maupun *child* kanan. Sedangkan *method getLeft()*, *getRight_2002()*,

dan *getData_2002()* digunakan untuk mengambil informasi yang tersimpan pada *node*. *Method setData_2002()* digunakan untuk mengubah nilai data pada *node*.

4. Membuat *Method Preorder Traversal*

void printPostorder_2002 (Node_2511532002 node_2002)

Method printPostorder_2002() digunakan untuk menampilkan isi *Binary Tree* menggunakan metode *Postorder Traversal*. *Traversal* dilakukan dengan urutan *Left Subtree*, *Right Subtree*, kemudian *Root*. *Method* ini menggunakan rekursi untuk mengunjungi seluruh *node* hingga mencapai *node* terdalam terlebih dahulu sebelum menampilkan data *node* induknya.

5. Membuat *Method Postorder Traversal*

void printPostorder_2002 (Node_2511532002 node_2002)

Method printPostorder_2002() digunakan untuk menampilkan isi *Binary Tree* menggunakan metode *Postorder Traversal*. *Traversal* dilakukan dengan urutan *Left Subtree*, *Right Subtree*, kemudian *Root*. *Method* ini menggunakan rekursi untuk mengunjungi seluruh *node* hingga mencapai *node* terdalam terlebih dahulu sebelum menampilkan data *node* induknya.

6. Membuat *Method Inorder Traversal*

void printInorder_2002(Node_2511532002 node_2002)

Method printInorder_2002() digunakan untuk menampilkan isi *Binary Tree* menggunakan metode *Inorder Traversal*. *Traversal* dilakukan dengan urutan *Left Subtree*, *Root*, kemudian *Right Subtree*. Metode ini sering digunakan untuk menampilkan data dalam urutan tertentu pada struktur pohon.

7. Membuat *Method Print Tree*

public String print()

public String print(String prefix, boolean isTail, String sb)

Method print() digunakan untuk menampilkan struktur *Binary Tree* dalam bentuk visual menyerupai pohon. *Method* ini memanfaatkan rekursi untuk menelusuri *child* kanan dan *child* kiri secara bertingkat, kemudian menampilkan posisi setiap *node* menggunakan karakter khusus sehingga hubungan antar *node* dapat terlihat dengan jelas.

2.2.3 Membuat Program *Btree_2511532002*

```
package pekan9_2511532002;

public class Btree_2511532002 {
    private Node_2511532002 root_2002;
    private Node_2511532002 currentNode_2002;
    public Btree_2511532002() {
        root_2002=null;
    }
    public boolean search_2002(int data_2002) {
        return search_2002(root_2002, data_2002);
    }
    private boolean search_2002(Node_2511532002 node_2002, int data_2002) {
        if (node_2002.getData_2002() ==data_2002)
            return true;
        if (node_2002.getLeft() != null)
            if (search_2002(node_2002.getLeft(), data_2002))
                return true;
        if (node_2002.getRight_2002() != null)
            if(search_2002(node_2002.getRight_2002(),data_2002))
                return true;
        return false;
    }
    public void printInorder_2002() {
        root_2002.printInorder_2002(root_2002);
    }
    public void printPreorder_2002() {
        root_2002.printPreorder_2002(root_2002);
    }
    public void printPostorder_2002() {
        root_2002.printPostorder_2002(root_2002);
    }
    public Node_2511532002 getRoot_2002() {
        return root_2002;
    }
    public boolean isEmpty_2002() {
        return root_2002==null;
    }
    public int countNodes_2002() {
        return countNodes_2002(root_2002);
    }
}
```

```

private int countNodes_2002 (Node_2511532002 node_2002) {
    int count_2002 =1;
    if (node_2002 == null) {
        return 0;
    }else {
        count_2002 +=countNodes_2002(node_2002.getLeft());
        count_2002 += countNodes_2002(node_2002.getRight_2002());
        return count_2002;
    }
}
public void print() {
    root_2002.print();
}
public Node_2511532002 getCurrent_2002() {
    return currentNode_2002;
}
public void setCurrent_2002 (Node_2511532002 node_2002)
{
    this.currentNode_2002=node_2002;
}
public void setRoot_2002 (Node_2511532002 root_2002) {
    this.root_2002=root_2002;
}
}

```

1. Membuat class *Btree_2511532002*

```
public class Btree_2511532002 {
```

Syntax di atas digunakan untuk membuat *class* bernama *Btree_2511532002*. Untuk merepresentasikan struktur *Binary Tree* secara keseluruhan. *Class* ini memiliki atribut *root_2002* yang berfungsi sebagai akar pohon (*root node*) dan *currentNode_2002* yang digunakan untuk menyimpan *node* yang sedang aktif atau sedang diproses.

2. Membuat *Constructor Btree*

```
public Btree_2511532002() {
    root_2002 = null;
}

```

Constructor Btree_2511532002() digunakan untuk membuat objek *Binary Tree* baru. Pada saat *tree* pertama kali dibuat, nilai *root* diinisialisasi dengan *null* yang menandakan bahwa pohon masih kosong dan belum memiliki simpul.

3. Membuat *Method* Pencarian Data (*Search*)

```
public boolean search_2002(int data_2002)

private boolean search_2002(Node_2511532002 node_2002,

```

int data_2002)

Method search_2002() digunakan untuk mencari keberadaan suatu data pada *Binary Tree*. Proses pencarian dilakukan secara rekursif dengan memeriksa *node* saat ini, kemudian menelusuri subtree kiri dan subtree kanan hingga data ditemukan atau seluruh *node* selesai diperiksa. *Method* ini akan mengembalikan nilai *true* jika data ditemukan dan *false* jika data tidak ditemukan.

4. Membuat *Method Traversal Tree*

```
public void printInorder_2002() {
    root_2002.printInorder_2002(root_2002);
}
public void printPreorder_2002() {
    root_2002.printPreorder_2002(root_2002);
}
public void printPostorder_2002() {
    root_2002.printPostorder_2002(root_2002);
}
```

Ketiga *method* tersebut digunakan untuk menampilkan isi *Binary Tree* menggunakan metode traversal yang berbeda. *Method printInorder_2002()* menampilkan data dengan urutan *Inorder*, *printPreorder_2002()* menampilkan data dengan urutan *Preorder*, dan *printPostorder_2002()* menampilkan data dengan urutan *Postorder*. Seluruh proses *traversal* dijalankan melalui *method* yang telah dibuat pada *class Node_2511532002*.

5. Membuat *Method Pemeriksaan Tree Kosong*

```
public boolean isEmpty_2002()
```

Method isEmpty_2002() digunakan untuk memeriksa apakah *Binary Tree* masih kosong atau sudah memiliki *root*. *Method* ini

akan mengembalikan nilai *true* jika *root* bernilai *null*, dan *false* jika *tree* telah berisi data.

6. Membuat *Method* Menghitung Jumlah Simpul

```
public int countNodes_2002() {
    return countNodes_2002(root_2002);
}

private int countNodes_2002 (Node_2511532002 node_2002) {
    int count_2002 =1;
    if (node_2002 == null) {
        return 0;
    }else {
        count_2002 += countNodes_2002(node_2002.getLeft());
        count_2002 += countNodes_2002
(node_2002.getRight_2002());
        return count_2002;
    }
}
```

Method *countNodes_2002()* digunakan untuk menghitung jumlah seluruh simpul yang terdapat pada *Binary Tree*. Perhitungan dilakukan secara rekursif dengan menjumlahkan node saat ini, *node* pada *subtree* kiri, dan *node* pada *subtree* kanan hingga seluruh simpul selesai dihitung.

7. Membuat *Method* Menampilkan Struktur Tree

```
public void print() {
    root_2002.print();
}
```

Method *print()* digunakan untuk menampilkan struktur *Binary Tree* dalam bentuk visual. *Method* ini memanggil *method* *print()* yang terdapat pada *class* *Node_2511532002* sehingga susunan *parent* dan *child* pada pohon dapat terlihat dengan lebih jelas.

8. Membuat *Method Getter dan Setter*

```
public Node_2511532002 getCurrent_2002() {
    return currentNode_2002;
}

public void setCurrent_2002 (Node_2511532002 node_2002) {
    this.currentNode_2002=node_2002;
}

public void setRoot_2002 (Node_2511532002 root_2002) {
    this.root_2002=root_2002;
}
}
```

Method getter dan *setter* digunakan untuk mengambil serta mengatur nilai atribut yang dimiliki oleh *Binary Tree*. *Method* *getRoot_2002()* digunakan untuk memperoleh *root tree*, sedangkan *setRoot_2002()* digunakan untuk menetapkan *root* baru. *Method* *getCurrent_2002()* dan *setCurrent_2002()* digunakan untuk mengakses maupun mengubah *node* yang sedang aktif pada proses pengolahan tree.

2.2.4 Membuat Program *BtreeDriver_2511532002* {

```
package pekan9_2511532002;

public class BtreeDriver_2511532002 {
    public static void main(String[] args) {

        //Membuat pohon
        Btree_2511532002 tree_2002 = new Btree_2511532002();
        System.out.println("Jumlah Simpul Awal Pohon: ");
        System.out.println(tree_2002.countNodes_2002());

        //Menambahkan simpul data 1
        Node_2511532002 root_2002=new Node_2511532002(1);

        //Menjadikan simpul 1 sebagai root
        tree_2002.setRoot_2002(root_2002);
        System.out.println("Jumlah simpul jika hanya ada root ");
        System.out.println(tree_2002.countNodes_2002());
        Node_2511532002 node2 =new Node_2511532002(2);
        Node_2511532002 node3 =new Node_2511532002(3);
        Node_2511532002 node4 =new Node_2511532002(4);
        Node_2511532002 node5 =new Node_2511532002(5);
        Node_2511532002 node6 =new Node_2511532002(6);
        Node_2511532002 node7 =new Node_2511532002(7);
        Node_2511532002 node8 =new Node_2511532002(8);
        Node_2511532002 node9 =new Node_2511532002(9);

        root_2002.setLeft_2002(node2);
        node2.setLeft_2002(node4);
```

```

        node2.setRight_2002(node5);
        node4.setRight_2002(node8);
        root_2002.setRight_2002(node3);
        node3.setLeft_2002(node6);
        node3.setRight_2002(node7);
        node6.setLeft_2002(node9);

        //Set root
        tree_2002.setCurrent_2002(tree_2002.getRoot_2002());
        System.out.println("Menampilkan simpul terakhir: ");
        System.out.println(tree_2002.getCurrent_2002().getData_2002());
        System.out.println("Jumlah simpul; setelah simpul 7
ditambahkan");
        System.out.println(tree_2002.countNodes_2002());
        System.out.println("InOrder: ");
        tree_2002.printInorder_2002();
        System.out.println("\nPreorder: ");
        tree_2002.printPreorder_2002();
        System.out.println("\nPostorder: ");
        tree_2002.printPostorder_2002();
        System.out.println("\nD Menampilkan simpul dalam bentuk pohon: ");
        tree_2002.print();
    }
}

```

1. Membuat class Driver Binary Tree

```
public class BinaryTree_2511532002 {
```

Class *BtreeDriver_2511532002* berfungsi sebagai program utama untuk menjalankan dan menguji seluruh operasi yang terdapat pada *Binary Tree*. Pada class ini dilakukan pembuatan objek *tree*, penambahan simpul, pembentukan struktur pohon, perhitungan jumlah simpul, serta pengujian *traversal* dan tampilan struktur pohon.

2. Membuat Objek Binary Tree

```
Btree_2511532002 tree_2002 = new Btree_2511532002();
```

```
System.out.println("Jumlah Simpul Awal Pohon: ");
```

```
System.out.println(tree_2002.countNodes_2002());
```

Kode tersebut digunakan untuk membuat objek *Binary Tree* baru yang masih kosong. Setelah tree dibuat, program menampilkan jumlah simpul awal menggunakan method *countNodes_2002()* untuk membuktikan bahwa *tree* belum memiliki *node*.

3. Membuat *Root Tree*

```
Node_2511532002 root_2002 = new Node_2511532002(1);
tree_2002.setRoot_2002(root_2002);
```

Kode tersebut digunakan untuk membuat simpul dengan nilai data 1 yang akan dijadikan sebagai *root* dari *Binary Tree*. Setelah *root* dibuat, *method setRoot_2002()* digunakan untuk menetapkan simpul tersebut sebagai akar pohon.

4. Membuat Simpul-simpul *Tree*

```
Node_2511532002 node2 = new Node_2511532002(2);
Node_2511532002 node3 = new Node_2511532002(3);
Node_2511532002 node4 = new Node_2511532002(4);
Node_2511532002 node5 = new Node_2511532002(5);
Node_2511532002 node6 = new Node_2511532002(6);
Node_2511532002 node7 = new Node_2511532002(7);
Node_2511532002 node8 = new Node_2511532002(8);
Node_2511532002 node9 = new Node_2511532002(9);
```

Kode tersebut digunakan untuk membuat beberapa objek *node* yang akan menjadi bagian dari *Binary Tree*. Setiap *node* memiliki nilai data yang berbeda dan nantinya akan dihubungkan untuk membentuk struktur pohon.

5. Membentuk Struktur *Binary Tree*

```
root_2002.setLeft_2002(node2);
node2.setLeft_2002(node4);
node2.setRight_2002(node5);
node4.setRight_2002(node8);

root_2002.setRight_2002(node3);
node3.setLeft_2002(node6);
node3.setRight_2002(node7);
node6.setLeft_2002(node9);
```


Kode tersebut digunakan untuk menghubungkan setiap *node* sehingga terbentuk struktur *Binary Tree*. Hubungan antar node dibangun menggunakan *method* *setLeft_2002()* dan *setRight_2002()* yang berfungsi untuk menentukan *child* kiri dan *child* kanan dari setiap simpul.

6. Menentukan *Current Node*

```
tree_2002.setCurrent_2002(tree_2002.getRoot_2002());
System.out.println("Menampilkan simpul terakhir: ");
System.out.println(tree_2002.getCurrent_2002().getData_2002());
```

Kode tersebut digunakan untuk mengatur *node* yang sedang aktif (*current node*). Pada program ini *current node* diatur menjadi *root tree*. Selanjutnya data yang terdapat pada *current node* ditampilkan ke layar.

7. Menghitung Jumlah Simpul

```
System.out.println("Jumlah simpul setelah simpul ditambahkan");
System.out.println(tree_2002.countNodes_2002());
```

Kode tersebut digunakan untuk menghitung jumlah seluruh simpul yang terdapat pada *Binary Tree* setelah proses pembentukan pohon selesai dilakukan. Perhitungan dilakukan menggunakan *method* *countNodes_2002()*.

8. Menampilkan *Traversal Binary Tree*

```
System.out.println("InOrder: ");
tree_2002.printInorder_2002();

System.out.println("\nPreorder: ");
tree_2002.printPreorder_2002();

System.out.println("\nPostorder: ");
tree_2002.printPostorder_2002();
```

Kode tersebut digunakan untuk menampilkan isi *Binary Tree* menggunakan tiga metode *traversal*, yaitu *Inorder*, *Preorder*, dan *Postorder*. Setiap traversal memiliki urutan kunjungan *node* yang berbeda sehingga menghasilkan urutan *output* yang berbeda pula.

9. Menampilkan Struktur Pohon

```
System.out.println("\nMenampilkan simpul dalam bentuk
pohon: "); tree_2002.print();
```

Kode tersebut digunakan untuk menampilkan seluruh struktur *Binary Tree* dalam bentuk visual menyerupai pohon. Tampilan ini memudahkan pengguna dalam memahami hubungan antara *root*, *child* kiri, dan *child* kanan pada setiap *node* yang terdapat dalam *tree*.

2.2.5 Membuat Program *GraphTravesal_2511532002*

```
package pekan9_2511532002;
import java.util.*;

public class GraphTravesal_2511532002 {

    private Map<String, List<String>> graph_2002 = new HashMap<>();

    public void addEdge_2002(String node1_2002, String node2_2002) {
        graph_2002.putIfAbsent(node1_2002, new ArrayList<>());
        graph_2002.putIfAbsent(node2_2002, new ArrayList<>());

        graph_2002.get(node1_2002).add(node2_2002);
        graph_2002.get(node2_2002).add(node1_2002);
    }

    public void printGraph_2002() {
        System.out.println("Graf Awal (Adjacency List):");

        for (String node_2002 : graph_2002.keySet()) {
            System.out.print(node_2002 + " -> ");

            List<String> neighbors_2002 = graph_2002.get(node_2002);
            System.out.println(String.join(", ", neighbors_2002));
        }

        System.out.println();
    }

    public void dfs_2002(String start_2002) {
        Set<String> visited_2002 = new HashSet<>();
        System.out.println("Penelusuran DFS:");
    }
}
```

```

        dfsHelper_2002(start_2002, visited_2002);

        System.out.println();
    }

    private void dfsHelper_2002(String current_2002,
                                Set<String> visited_2002) {

        if (visited_2002.contains(current_2002))
            return;

        visited_2002.add(current_2002);

        System.out.print(current_2002 + " ");

        for (String neighbor_2002 :
             graph_2002.getOrDefault(current_2002,
                                     new ArrayList<>())) {

            dfsHelper_2002(neighbor_2002, visited_2002);
        }
    }

    public void bfs_2002(String start_2002) {

        Set<String> visited_2002 = new HashSet<>();
        Queue<String> queue_2002 = new LinkedList<>();

        queue_2002.add(start_2002);
        visited_2002.add(start_2002);

        System.out.println("Penelusuran BFS:");

        while (!queue_2002.isEmpty()) {

            String current_2002 = queue_2002.poll();

            System.out.print(current_2002 + " ");

            for (String neighbor_2002 :
                 graph_2002.getOrDefault(current_2002,
                                         new ArrayList<>())) {

                if (!visited_2002.contains(neighbor_2002)) {

                    queue_2002.add(neighbor_2002);
                    visited_2002.add(neighbor_2002);
                }
            }
        }

        System.out.println();
    }

    public static void main(String[] args) {

        GraphTraversal_2511532002 graph_2002 =
            new GraphTraversal_2511532002();

        graph_2002.addEdge_2002("A", "B");
        graph_2002.addEdge_2002("A", "C");
        graph_2002.addEdge_2002("B", "D");
        graph_2002.addEdge_2002("B", "E");

        System.out.println("Graf Awal adalah:");
        graph_2002.printGraph_2002();

        graph_2002.dfs_2002("A");
        graph_2002.bfs_2002("A");
    }
}

```

1. Membuat *Class Graph Traversal*

Menulis syntax program, untuk menulis *syntax* buka tab “*Source*” maka akan terbuka tab *syntax* program *InsertionSortGUI_2511532002*.

2. Membuat Struktur *Graph*

```
private Map<String, List<String>> graph_2002 = new
HashMap<>();
```

Kode tersebut digunakan untuk merepresentasikan *graph* menggunakan struktur *Adjacency List*. Setiap simpul disimpan sebagai *key* pada *HashMap*, sedangkan daftar simpul yang terhubung dengannya disimpan dalam bentuk *List*. Representasi ini membuat penyimpanan *graph* menjadi lebih efisien dan mudah dikelola.

3. Membuat *Method* Menambahkan *Edge*

```
public void addEdge_2002(String node1_2002,
String node2_2002)
```

Method addEdge_2002() digunakan untuk menambahkan hubungan (*edge*) antara dua simpul pada *graph*. Sebelum *edge* ditambahkan, program memastikan bahwa kedua simpul sudah tersedia pada *adjacency list*. Karena *graph* yang digunakan bersifat tidak berarah (*undirected graph*), maka setiap simpul akan saling ditambahkan ke daftar tetangga masing-masing.

4. Membuat *Method* Menampilkan *Graph*

```
public void printGraph_2002()
```

Method printGraph_2002() digunakan untuk menampilkan isi *graph* dalam bentuk *adjacency list*. *Method* ini melakukan perulangan pada seluruh simpul yang terdapat dalam *graph*, kemudian menampilkan daftar tetangga yang terhubung dengan simpul tersebut sehingga struktur *graph* dapat dilihat dengan jelas.

5. Membuat *Method* DFS (*Depth First Search*)

```
public void dfs_2002(String start_2002)
```

Method dfs_2002() digunakan untuk melakukan traversal graph menggunakan algoritma *Depth First Search (DFS)*. Proses traversal dimulai dari simpul awal yang diberikan, kemudian *method* ini membuat kumpulan simpul yang telah dikunjungi (*visited*) dan memanggil *method* rekursif *dfsHelper_2002()* untuk melakukan penelusuran hingga ke simpul terdalam.

6. Membuat *Method DFS Helper*

```
public void dfs_2002(String start_2002)
```

Method dfs_2002() digunakan untuk melakukan *traversal graph* menggunakan algoritma *Depth First Search (DFS)*. Proses *traversal* dimulai dari simpul awal yang diberikan, kemudian *method* ini membuat kumpulan simpul yang telah dikunjungi (*visited*) dan memanggil *method* rekursif *dfsHelper_2002()* untuk melakukan penelusuran hingga ke simpul terdalam.

7. Membuat *Method BFS (Breadth First Search)*

```
public void bfs_2002(String start_2002)
```

Method bfs_2002() digunakan untuk melakukan traversal graph menggunakan algoritma *Breadth First Search (BFS)*. Algoritma ini memanfaatkan struktur data *Queue* untuk menyimpan simpul yang akan diproses. Penelusuran dilakukan secara bertahap berdasarkan tingkat kedekatan simpul terhadap simpul awal sehingga seluruh simpul pada level yang sama akan dikunjungi terlebih dahulu sebelum berpindah ke level berikutnya.

8. Membuat *Method Main*

```
public static void main(String[] args)
```

Method main() digunakan sebagai program utama untuk menjalankan implementasi graph. Pada *method* ini dibuat objek graph baru, kemudian ditambahkan beberapa *edge* menggunakan *method addEdge_2002()*. Setelah graph terbentuk, program menampilkan *adjacency list graph* dan menjalankan *traversal DFS* serta *BFS* yang dimulai dari simpul A.

9. Membentuk Struktur *Graph*

```
graph_2002.addEdge_2002("A", "B");
graph_2002.addEdge_2002("A", "C");
graph_2002.addEdge_2002("B", "D");
graph_2002.addEdge_2002("B", "E");
```

Kode tersebut digunakan untuk membangun *graph* dengan menambahkan hubungan antar simpul.

10. Menampilkan dan Melakukan *Traversal Graph*

```
System.out.println("Graf Awal adalah:");
graph_2002.printGraph_2002();
```

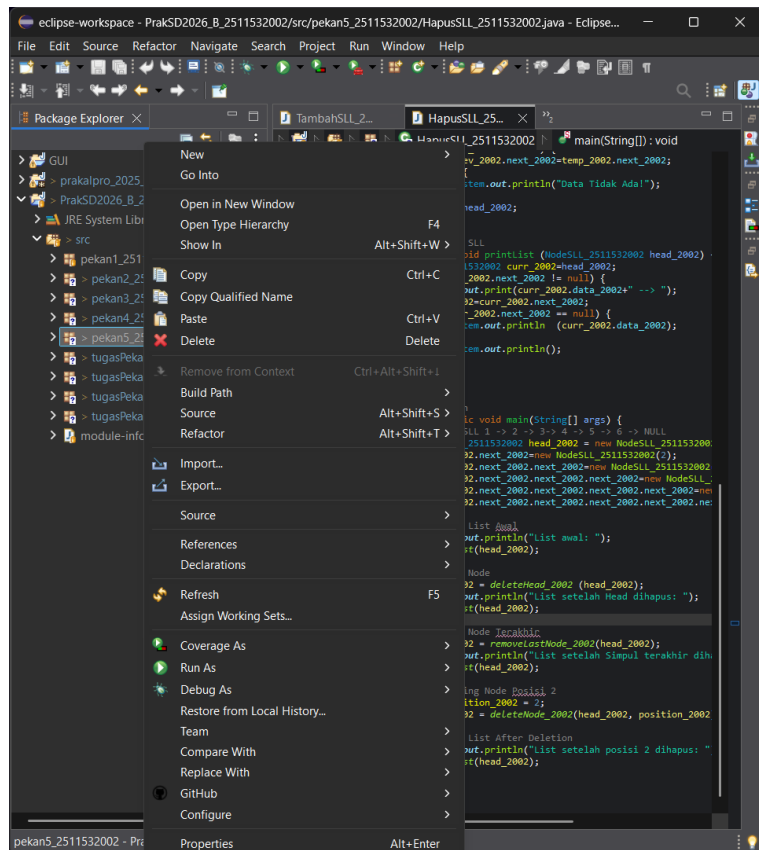
```
graph_2002.dfs_2002("A");
graph_2002.bfs_2002("A");
```

Kode tersebut digunakan untuk menampilkan *graph* dalam bentuk *adjacency list* dan melakukan *traversal* menggunakan algoritma *DFS* dan *BFS*. *Traversal* dimulai dari simpul A sehingga pengguna dapat melihat perbedaan urutan kunjungan simpul antara kedua algoritma tersebut.

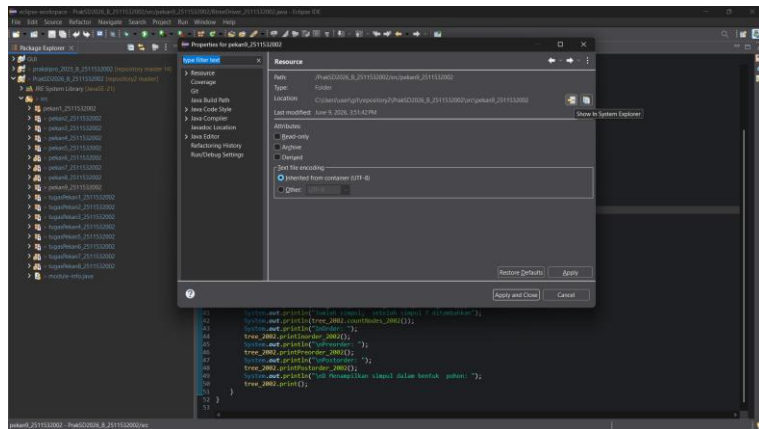
2.2.6 Upload ke GitHub

1. Setelah semua program selesai dibuat, selanjutnya adalah *user* perlu untuk memasukkan program yang dibuat di Eclipse ke *repository* GitHub. Selain dengan cara “*Team*” atau “*Commit and Push*”.

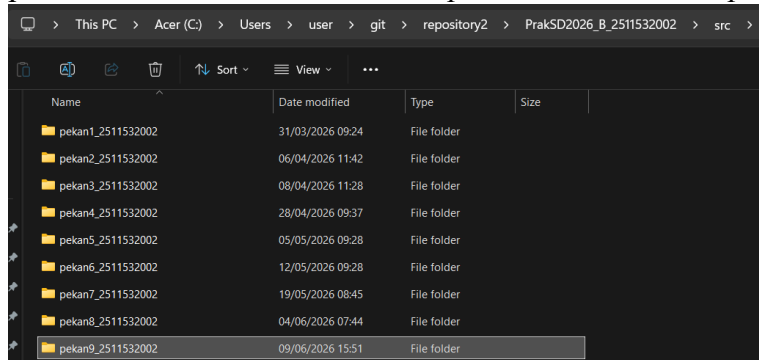
Bisa juga dengan cara *upload* manual. Caranya klik kanan pada *package*, lalu pilih *properties*.



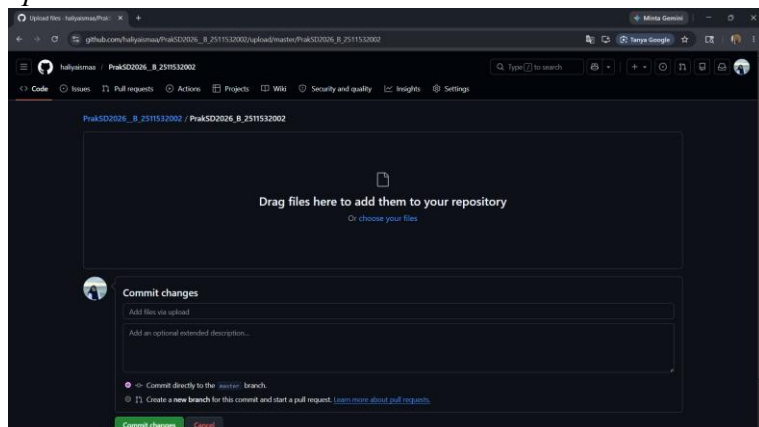
2. Maka akan muncul *pop up menu* “*Properteis for pekan9_2511532002*”. Lalu klik icon “*Show in System Folder*”.



3. Maka akan menunjukan lokasi file “*pekan9_2511532002*” pada *File Explorer computer*.

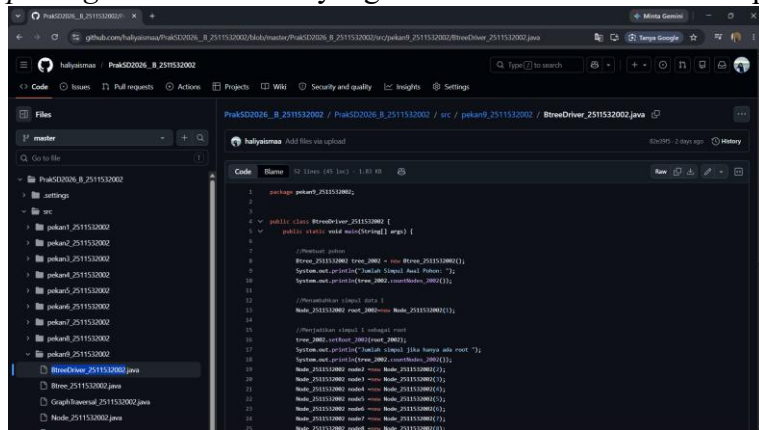


4. Setelah itu *drag and drop* file “*pekan9_2511532002*” ke GitHub. Lalu tambahkan keterangan jika perlu. Setelah itu klik *upload*.



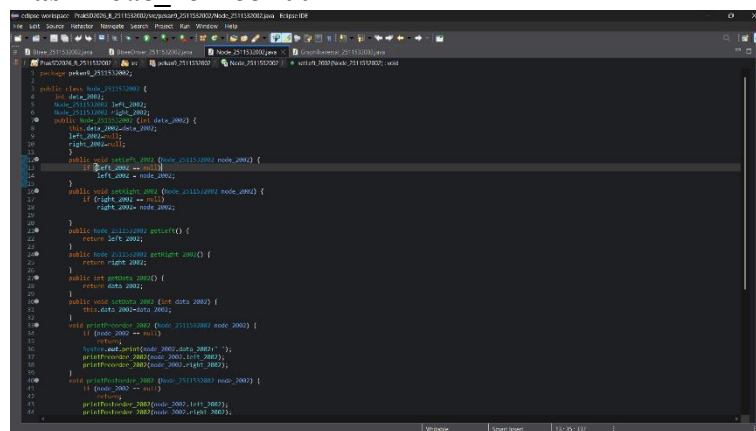
5. Maka dapat dilihat pada *repository GitHub* bahwa file “*pekan8_2511532002*” telah terunggah via *Upload*..

Dapat kita lihat disini jika kita buka, maka akan muncul *file package* dan *class* yang telah di buat di Eclipse.

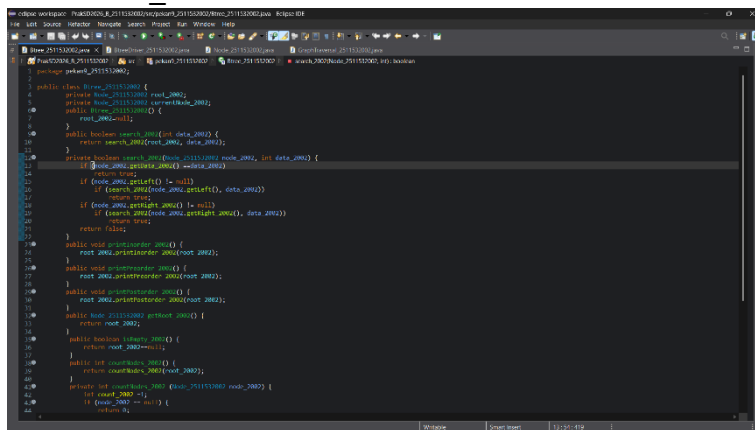


2.3 Hasil

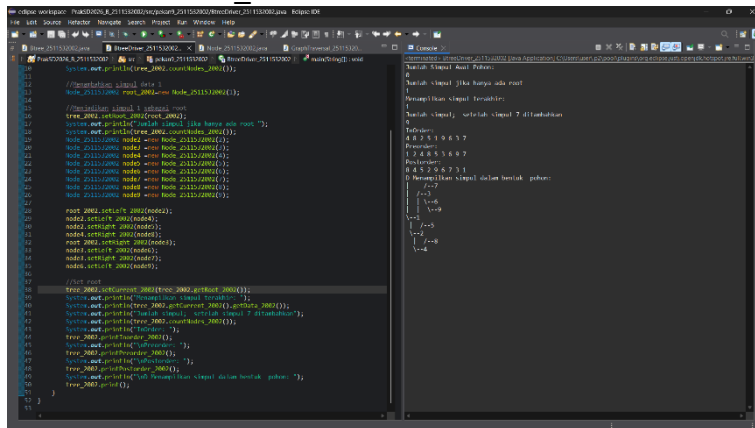
2.3.1 Hasil Node 2511532002



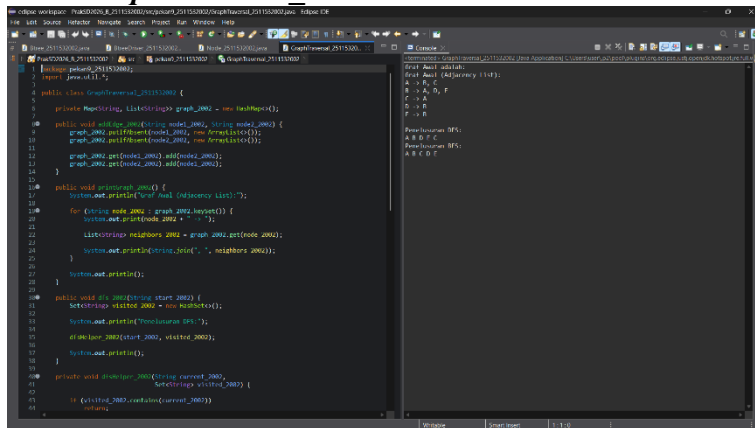
2.3.2 Hasil *Btree_2511532002*



2.3.3 Hasil *BtreeDriver*_2511532002



2.3.4 Hasil *GraphTraversal* 2511532002



BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data *Binary Tree* dan *Graph* berhasil diimplementasikan menggunakan bahasa pemrograman *Java*. Pada implementasi *Binary Tree*, program mampu membentuk struktur pohon, menghitung jumlah simpul, serta melakukan *traversal* menggunakan metode *Inorder*, *Preorder*, dan *Postorder* untuk menampilkan data dengan urutan yang berbeda.

Selain itu, implementasi *Graph* menggunakan representasi *Adjacency List* juga berhasil dilakukan. Program dapat membentuk hubungan antar simpul serta melakukan proses *traversal* menggunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS). Hasil *traversal* menunjukkan bahwa *DFS* melakukan penelusuran hingga ke simpul terdalam terlebih dahulu, sedangkan *BFS* menelusuri simpul berdasarkan tingkat atau level kedekatannya dengan simpul awal.

Melalui praktikum ini, pemahaman mengenai konsep struktur data *non-linear*, khususnya *Binary Tree* dan *Graph*, serta teknik *traversal* yang digunakan pada kedua struktur data tersebut menjadi lebih baik. Praktikum ini juga memberikan pengalaman dalam mengimplementasikan algoritma *traversal* menggunakan bahasa pemrograman *Java*.

3.2 Saran

Saran untuk praktikum selanjutnya adalah agar mahasiswa lebih teliti dalam memahami konsep *Binary Tree* dan *Graph* serta proses *traversal* menggunakan metode *Inorder*, *Preorder*, *Postorder*, *DFS*, dan *BFS*.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Binary Tree Data Structure," 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/binary-tree-data-structure/>. Diakses pada: 11 Juni 2026.
- [2] GeeksforGeeks, "Introduction to Binary Tree," 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/introduction-to-binary-tree/>. Diakses pada: 11 Juni 2026.
- [3] GeeksforGeeks, "Inorder Traversal of Binary Tree," 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/inorder-traversal-of-binary-tree/>. Diakses pada: 11 Juni 2026.
- [4] GeeksforGeeks, "Preorder Traversal of Binary Tree," 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/preorder-traversal-of-binary-tree/>. Diakses pada: 11 Juni 2026.
- [5] GeeksforGeeks, "Postorder Traversal of Binary Tree," 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/postorder-traversal-of-binary-tree/>. Diakses pada: 11 Juni 2026.
- [6] GeeksforGeeks, "Level Order Tree Traversal," 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/level-order-tree-traversal/>. Diakses pada: 11 Juni 2026.
- [7] GeeksforGeeks, "BFS vs DFS for Binary Tree," 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/bfs-vs-dfs-binary-tree/>. Diakses pada: 11 Juni 2026.

TUGAS PRAKTIKUM PEKAN 5

1. Kode Program

1.1 Graph_2511532002

```
package tugasPekan9_2511532002;

import java.util.*;

public class Graph_2511532002 {
    private Map<String, List<String>> graph_2002;
    public Graph_2511532002() {
        graph_2002 = new HashMap<>();
    }

    // Menambahkan Edge
    public void addEdge_2002(String source_2002, String destination_2002) {
        graph_2002.putIfAbsent(source_2002, new ArrayList<>());
        graph_2002.putIfAbsent(destination_2002, new ArrayList<>());

        graph_2002.get(source_2002).add(destination_2002);
        graph_2002.get(destination_2002).add(source_2002);
    }

    // Mengambil Graph
    public Map<String, List<String>> getGraph_2002() {
        return graph_2002;
    }

    // BFS
    public List<String> bfs_2002(String start_2002,
        String goal_2002) {

        Queue<String> queue_2002 = new LinkedList<>();
        Set<String> visited_2002 = new HashSet<>();
        Map<String, String> parent_2002 = new HashMap<>();

        queue_2002.add(start_2002);
        visited_2002.add(start_2002);

        while (!queue_2002.isEmpty()) {
            String current_2002 = queue_2002.poll();

            if (current_2002.equals(goal_2002)) {
                break;
            }

            for (String neighbor_2002 :
                graph_2002.getOrDefault(current_2002,
                    new ArrayList<>())) {
                if (!visited_2002.contains(neighbor_2002)) {
                    visited_2002.add(neighbor_2002);
                    parent_2002.put(neighbor_2002,
                        current_2002);
                    queue_2002.add(neighbor_2002);
                }
            }
        }

        return buildPath_2002(parent_2002,
```

```

start_2002,
goal_2002);
}

// DFS
public List<String> dfs_2002(String start_2002,
String goal_2002) {

Stack<String> stack_2002 = new Stack<>();
Set<String> visited_2002 = new HashSet<>();
Map<String, String> parent_2002 = new HashMap<>();

stack_2002.push(start_2002);

while (!stack_2002.isEmpty()) {
String current_2002 = stack_2002.pop();

if (visited_2002.contains(current_2002)) {
continue;
}
visited_2002.add(current_2002);

if (current_2002.equals(goal_2002)) {
break;
}
List<String> neighbors_2002 =
graph_2002.getOrDefault(
current_2002,
new ArrayList<>());

for (int i_2002 = neighbors_2002.size() - 1;
i_2002 >= 0;
i_2002--) {

String neighbor_2002 =
neighbors_2002.get(i_2002);

if (!visited_2002.contains(neighbor_2002)) {

parent_2002.put(neighbor_2002,
current_2002);

stack_2002.push(neighbor_2002);
}
}
}
return buildPath_2002(parent_2002, start_2002, goal_2002);
}

// Membentuk Path
private List<String> buildPath_2002(
Map<String, String> parent_2002,
String start_2002,
String goal_2002) {

List<String> path_2002 =
new ArrayList<>();

if (!start_2002.equals(goal_2002) && !parent_2002.containsKey(goal_2002)) {

return path_2002;
}

String current_2002 = goal_2002;

while (current_2002 != null) {

path_2002.add(current_2002);

if (current_2002.equals(start_2002)) {

```

```

break;
}

current_2002 =
parent_2002.get(current_2002);
}

Collections.reverse(path_2002);
return path_2002;
}
}

```

Penjelasan:

1) Membuat *class Graph*

```

public class Graph_2511532002 {
    private Map<String, List<String>> graph_2002;
}

```

Syntax di atas digunakan untuk membuat *class Graph_2511532002* yang berfungsi sebagai representasi struktur data graf. Variabel *graph_2002* bertipe *Map<String, List<String>>* digunakan untuk menyimpan hubungan antar simpul dalam bentuk *adjacency list*, sehingga setiap simpul dapat memiliki daftar simpul tetangga yang terhubung dengannya.

2) Membuat *Constructor Graph*

```

public Graph_2511532002() {
    graph_2002 = new HashMap<>();
}

```

Syntax di atas digunakan untuk membuat *constructor Graph_2511532002()*.

Constructor ini berfungsi menginisialisasi variabel *graph_2002* menjadi objek *HashMap* sehingga graf siap digunakan untuk menyimpan data simpul dan hubungan antar simpul.

3) Membuat *Method Menambahkan Edge*

```

public void addEdge_2002(String source_2002, String
destination_2002) {

```

```

    graph_2002.putIfAbsent(source_2002, new ArrayList<>());
    graph_2002.putIfAbsent(destination_2002, new ArrayList<>());
}

```

```
graph_2002.get(source_2002).add(destination_2002);
graph_2002.get(destination_2002).add(source_2002);
}
```

Syntax di atas digunakan untuk membuat *method* *addEdge_2002()* yang berfungsi menambahkan hubungan (*edge*) antara dua simpul pada graf. *Method* ini juga memastikan setiap simpul memiliki daftar tetangga, kemudian menghubungkan kedua simpul sehingga dapat diakses dari dua arah pada graf tak berarah.

1.2 Kelas *GraphPanel_2511532002*

```
package tugasPekan9_2511532002;
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.Point;
import java.util.*;
import javax.swing.JPanel;

public class GraphPanel_2511532002 extends JPanel{

    private Map<String, Point> positions_2002;
    private Graph_2511532002 graph_2002;
    private Set<String> visitedNodes_2002;

    public GraphPanel_2511532002(Graph_2511532002 graph_2002) {

        this.graph_2002 = graph_2002;
        this.visitedNodes_2002 = new HashSet<>();

        positions_2002 = new HashMap<>();
        setBackground(Color.WHITE);

        // Posisi 10 node
        positions_2002.put("Air Tawar", new Point(300, 50));
        positions_2002.put("Ulak Karang", new Point(150, 120));
        positions_2002.put("Lubuk Buaya", new Point(450, 120));
        positions_2002.put("Purus", new Point(80, 220));
        positions_2002.put("Alai", new Point(220, 220));
        positions_2002.put("Tabing", new Point(380, 220));
        positions_2002.put("Koto Tengah", new Point(520, 220));
        positions_2002.put("Padang Barat", new Point(150, 320));
        positions_2002.put("Siteba", new Point(300, 320));
        positions_2002.put("By Pass", new Point(450, 320));

        setBackground(Color.WHITE);
    }

    public void setVisitedNodes_2002(
        Collection<String> visited_2002) {

        visitedNodes_2002.clear();
        visitedNodes_2002.addAll(visited_2002);

        repaint();
    }
}
```



```

}

public void resetGraph_2002() {

    visitedNodes_2002.clear();

    repaint();
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);

    Graphics2D g2_2002 = (Graphics2D) g;

    // gambar edge
    for (String node_2002 :
        graph_2002.getGraph_2002().keySet()) {

        Point p1_2002 =
            positions_2002.get(node_2002);

        for (String neighbor_2002 :
            graph_2002.getGraph_2002()
                .get(node_2002)) {

            Point p2_2002 =
                positions_2002.get(neighbor_2002);

            g2_2002.drawLine(
                p1_2002.x,
                p1_2002.y,
                p2_2002.x,
                p2_2002.y);

        }

    }

    // gambar node
    for (String node_2002 :
        positions_2002.keySet()) {

        Point p_2002 =
            positions_2002.get(node_2002);

        if (visitedNodes_2002.contains(node_2002)) {

            g2_2002.setColor(Color.GREEN);

        } else {

            g2_2002.setColor(Color.LIGHT_GRAY);

        }

        g2_2002.fillOval(
            p_2002.x - 40,
            p_2002.y - 40,
            75,
            75);

        g2_2002.setColor(Color.BLACK);

        g2_2002.drawOval(
            p_2002.x - 35,
            p_2002.y - 35,
            70,
            70);

        java.awt.FontMetrics fm_2002 =
            g2_2002.getFontMetrics();

        int width_2002 =

```

```

        fm_2002.stringWidth(node_2002);

        g2_2002.drawString(
            node_2002,
            p_2002.x - width_2002 / 2,
            p_2002.y + 5);
    }
}

```

Penjelasan:

1) *Method Class GraphPanel*

```
public class GraphPanel_2511532002 extends JPanel {
```

Syntax di atas digunakan untuk membuat *class* *GraphPanel_2511532002* yang merupakan turunan dari *JPanel*. *Class* ini berfungsi untuk menampilkan visualisasi graf berupa simpul (*node*) dan hubungan antar simpul (*edge*) pada antarmuka GUI.

2) *Membuat Atribut GraphPanel*

```
private Map<String, Point> positions_2002;
private Graph_2511532002 graph_2002;
private Set<String> visitedNodes_2002;
```

Syntax di atas digunakan untuk mendeklarasikan atribut yang diperlukan dalam visualisasi *graf*. Variabel *positions_2002* digunakan untuk menyimpan posisi setiap simpul, *graph_2002* digunakan untuk menyimpan data graf, dan *visitedNodes_2002* digunakan untuk menyimpan simpul yang telah dikunjungi saat proses *BFS* atau *DFS*.

3) *Membuat Constructor GraphPanel*

```
public GraphPanel_2511532002(Graph_2511532002 graph_2002)
```

Syntax di atas digunakan untuk membuat *constructor* *GraphPanel_2511532002*. *Constructor* ini berfungsi menginisialisasi objek graf, menentukan posisi setiap simpul pada panel, serta menyiapkan himpunan simpul yang akan digunakan untuk visualisasi proses penelusuran graf.

- 4) Membuat *Method SetVisitedNodes*
`public void setVisitedNodes_2002(
 Collection<String> visited_2002)`

Syntax di atas digunakan untuk membuat *method setVisitedNodes_2002()* yang berfungsi memperbarui daftar simpul yang telah dikunjungi. *Method* ini digunakan untuk menampilkan perubahan warna pada simpul yang telah dilalui oleh algoritma *BFS* atau *DFS*.

- 5) Membuat *Method ResetGraph*
`public void resetGraph_2002()`

Syntax di atas digunakan untuk membuat *method resetGraph_2002()* yang berfungsi menghapus seluruh data simpul yang telah dikunjungi sehingga tampilan graf kembali ke kondisi awal sebelum proses penelusuran dilakukan.

- 6) Membuat *Membuat PaintComponent*
`@Override
 protected void paintComponent(Graphics g)`

Syntax di atas digunakan untuk membuat *method paintComponent()* yang berfungsi menggambar seluruh komponen graf pada panel. *Method* ini digunakan untuk menampilkan edge sebagai garis penghubung antar simpul serta *node* sebagai lingkaran yang merepresentasikan lokasi atau vertex pada graf. Selain itu, *method* ini juga memberikan warna berbeda pada simpul yang telah dikunjungi selama proses *BFS* atau *DFS*.

1.3 SearchResult_2511532002

```
package tugasPekan9_2511532002;  
import java.util.*;  
  
public class SearchResult_2511532002 {
```

```

public static void main(String[] args) {
    class SearchResult_2002 {

        private List<String> path_2002;
        private List<String> visitOrder_2002;
        private int exploredNodes_2002;

        public SearchResult_2002(
            List<String> path_2002,
            List<String> visitOrder_2002,
            int exploredNodes_2002) {

            this.path_2002 = path_2002;
            this.visitOrder_2002 = visitOrder_2002;
            this.exploredNodes_2002=exploredNodes_2002;
        }

        public List<String> getPath_2002() {
            return path_2002;
        }

        public List<String> getVisitOrder_2002() {
            return visitOrder_2002;
        }

        public int getExploredNodes_2002() {
            return exploredNodes_2002;
        }

    }
}

```

Penjelasan:

- 1) Membuat Class *SearchResult*
`public class SearchResult_2511532002 {`

Syntax di atas digunakan untuk membuat *class* *SearchResult_2511532002* yang berfungsi menyimpan hasil proses pencarian jalur pada graf menggunakan algoritma *BFS* maupun *DFS*.

- 2) Membuat Atribut *SearchResult*
`private List<String> path_2002;`
`private List<String> visitedOrder_2002;`
`private int visitedCount_2002;`

Syntax di atas digunakan untuk mendeklarasikan atribut hasil pencarian. Variabel *path_2002* digunakan untuk menyimpan jalur yang ditemukan, *visitedOrder_2002* digunakan untuk menyimpan urutan simpul yang dikunjungi, sedangkan

visitedCount_2002 digunakan untuk menyimpan jumlah simpul yang telah dikunjungi selama proses pencarian.

3) Memuat *Constructor SearchResult*

```
public SearchResult_2511532002(
    List<String> path_2002,
    List<String> visitedOrder_2002,
    int visitedCount_2002)
```

Syntax di atas digunakan untuk membuat *constructor SearchResult_2511532002* yang berfungsi menginisialisasi data hasil pencarian berupa jalur yang ditemukan, urutan kunjungan simpul, dan jumlah simpul yang telah dikunjungi.

4) Membuat *Method Getter*

```
public List<String> getPath_2002()
public List<String> getVisitedOrder_2002()
public int getVisitedCount_2002()
```

Syntax di atas digunakan untuk membuat *method getter* yang berfungsi mengambil dan mengembalikan nilai atribut *path_2002*, *visitedOrder_2002*, dan *visitedCount_2002* sehingga dapat digunakan pada class lain, seperti GUI untuk menampilkan hasil *BFS* atau *DFS*.

1.4 PetaKampus_2511532002

```
package tugasPekan9_2511532002;

import java.awt.*;
import javax.swing.*;
import java.util.*;
import java.util.List;

public class PetaKampus_2511532002 extends JFrame {

    private static final long serialVersionUID = 1L;

    private Graph_2511532002 graph_2002;
    private GraphPanel_2511532002 graphPanel_2002;

    private JComboBox<String> startBox_2002;
    private JComboBox<String> goalBox_2002;
```

```

private JButton bfsButton_2002;
private JButton dfsButton_2002;
private JButton resetButton_2002;

private JTextArea resultArea_2002;

public PetaKampus_2511532002() {

    setTitle("BFS dan DFS - 2511532002");
    setSize(800, 700);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLayout(new BorderLayout());

    // Membuat Graph
    graph_2002 = new Graph_2511532002();

    // 10 Node dan 15 Edge
    graph_2002.addEdge_2002("Air Tawar", "Ulak Karang");
    graph_2002.addEdge_2002("Air Tawar", "Lubuk Buaya");
    graph_2002.addEdge_2002("Ulak Karang", "Purus");
    graph_2002.addEdge_2002("Ulak Karang", "Alai");
    graph_2002.addEdge_2002("Lubuk Buaya", "Tabing");
    graph_2002.addEdge_2002("Lubuk Buaya", "Koto Tengah");
    graph_2002.addEdge_2002("Purus", "Padang Barat");
    graph_2002.addEdge_2002("Alai", "Padang Barat");
    graph_2002.addEdge_2002("Alai", "Siteba");
    graph_2002.addEdge_2002("Tabing", "Siteba");
    graph_2002.addEdge_2002("Tabing", "By Pass");
    graph_2002.addEdge_2002("Koto Tengah", "By Pass");
    graph_2002.addEdge_2002("Siteba", "By Pass");

    // Panel Graph
    graphPanel_2002 = new GraphPanel_2511532002(graph_2002);

    // Pilihan Node
    // JComboBox
    String[] lokasi_2002 = {
        "Air Tawar",
        "Ulak Karang",
        "Lubuk Buaya",
        "Purus",
        "Alai",
        "Tabing",
        "Koto Tengah",
        "Padang Barat",
        "Siteba",
        "By Pass"
    };

    startBox_2002 = new JComboBox<>(lokasi_2002);
    goalBox_2002 = new JComboBox<>(lokasi_2002);

    bfsButton_2002 = new JButton("BFS");
    dfsButton_2002 = new JButton("DFS");
    resetButton_2002 = new JButton("Reset");

    resultArea_2002 = new JTextArea();
    resultArea_2002.setEditable(false);

    // Panel Atas
    JPanel topPanel_2002 = new JPanel();

    topPanel_2002.setLayout(new FlowLayout());

    topPanel_2002.add(startBox_2002);
    topPanel_2002.add(goalBox_2002);
    topPanel_2002.add(bfsButton_2002);
    topPanel_2002.add(dfsButton_2002);
    topPanel_2002.add(resetButton_2002);

```

```

// Event BFS
bfsButton_2002.addActionListener(e -> {

    String start_2002 =
        (String) startBox_2002.getSelectedItemAt();

    String goal_2002 =
        (String) goalBox_2002.getSelectedItemAt();

    List<String> path_2002 = graph_2002.bfs_2002( start_2002,
goal_2002);

    graphPanel_2002.setVisitedNodes_2002(path_2002);

    resultArea_2002.setText("");

    resultArea_2002.append("METODE : BFS\n\n");
    resultArea_2002.append("Start : " + start_2002 + "\n");
    resultArea_2002.append("Goal : " + goal_2002 + "\n\n");

    if(path_2002.isEmpty()) {
        resultArea_2002.append("Path tidak ditemukan");
    }else {
        resultArea_2002.append(
            "Path : "
            + String.join(" -> ",
path_2002));
    }
});

// Event DFS
dfsButton_2002.addActionListener(e -> {

    String start_2002 =
        (String) startBox_2002.getSelectedItemAt();

    String goal_2002 =
        (String) goalBox_2002.getSelectedItemAt();

    List<String> path_2002
=graph_2002.dfs_2002(start_2002,goal_2002);

    graphPanel_2002.setVisitedNodes_2002(path_2002);

    resultArea_2002.setText("");

    resultArea_2002.append("METODE : DFS\n\n");
    resultArea_2002.append("Start : " + start_2002 + "\n");
    resultArea_2002.append("Goal : " + goal_2002 + "\n\n");

    if(path_2002.isEmpty()) {
        resultArea_2002.append("Path tidak ditemukan");
    }else {
        resultArea_2002.append(
            "Path : "
            + String.join(" -> ",
path_2002));
    }
});

// Event Reset
resetButton_2002.addActionListener(e -> {

    graphPanel_2002.resetGraph_2002();
    resultArea_2002.setText("");

});

add(topPanel_2002, BorderLayout.NORTH);
add(graphPanel_2002, BorderLayout.CENTER);
add(new JScrollPane(resultArea_2002),

```

```

                                BorderLayout.SOUTH);
    }
    public static void main(String[] args) {
        new PetaKampus_2511532002().setVisible(true);
    }
}

```

Penjelasan:

- 1) Membuat *Class PetaKampus*
`public class PetaKampus_2511532002 extends JFrame {`

Syntax di atas digunakan untuk membuat *class PetaKampus_2511532002* yang merupakan turunan dari *JFrame*. *Class* ini berfungsi sebagai antarmuka (*GUI*) utama yang digunakan untuk menampilkan visualisasi graf serta menjalankan algoritma *BFS* dan *DFS*.

- 2) Membuat *Constructor PetaKampus*
`public PetaKampus_2511532002() {`

Syntax di atas digunakan untuk membuat constructor *PetaKampus_2511532002*. *Constructor* ini berfungsi mengatur tampilan *JFrame*, membuat objek graf, menambahkan simpul dan *edge*, serta menginisialisasi seluruh komponen *GUI* yang digunakan dalam program.

- 3) Membuat *Data Graph*
`graph_2002 = new Graph_2511532002();`
`graph_2002.addEdge_2002("Air Tawar", "Ulak Karang");`
`graph_2002.addEdge_2002("Air Tawar", "Lubuk Buaya");`
 ...

Syntax di atas digunakan untuk membuat objek graf dan menambahkan hubungan antar lokasi. Data ini digunakan sebagai representasi peta sederhana yang nantinya akan ditelusuri menggunakan algoritma *BFS* dan *DFS*.

- 4) Membuat *ComboBox Dropdown Lokasi*
`startBox_2002 = new JComboBox<>(lokasi_2002);`
`goalBox_2002 = new JComboBox<>(lokasi_2002);`

Syntax di atas digunakan untuk membuat komponen *JComboBox* yang berfungsi memilih lokasi awal (*start*) dan lokasi tujuan (*goal*) yang akan digunakan pada proses pencarian jalur.

- 5) Membuat Tombol *BFS*, *DFS*, dan *Reset*
`bfsButton_2002 = new JButton("BFS");`
`dfsButton_2002 = new JButton("DFS");`
`resetButton_2002 = new JButton("Reset");`

Syntax di atas digunakan untuk membuat tombol *BFS*, *DFS*, dan *Reset*. Tombol *BFS* digunakan untuk menjalankan algoritma *Breadth First Search*, tombol *DFS* digunakan untuk menjalankan algoritma *Depth First Search*, sedangkan tombol *Reset* digunakan untuk mengembalikan tampilan graf ke kondisi awal.

- 6) Membuat *Event BFS* dan *DFS*
`bfsButton_2002.addActionListener(e -> { ... });`
`dfsButton_2002.addActionListener(e -> { ... });`

Syntax di atas digunakan untuk membuat *event* pada tombol *BFS* dan *DFS*. Ketika tombol ditekan, program akan mengambil lokasi awal dan tujuan yang dipilih pengguna, menjalankan algoritma pencarian, kemudian menampilkan jalur yang ditemukan pada area hasil dan visualisasi graf.

- 7) Membuat *Event Reset*
`resetButton_2002.addActionListener(e -> { ... });`

Syntax di atas digunakan untuk membuat *event* pada tombol *Reset* yang berfungsi menghapus hasil pencarian dan mengembalikan warna node pada graf ke kondisi semula.

-
- The screenshot displays a Jupyter Notebook environment. The top toolbar includes standard file and editing icons, along with a search bar. The notebook contains a code cell with a Cypher query and a corresponding graph visualization.
- Cypher Query:**
- ```

124 (t1) MATCH (a0) WHERE (a0.id = 1);
125 LISTCOLUMNS path_2002_wprp_2002_dfs_2002(start_2002_qual_2002);
126 GRAPH_2002_RETURNS_NODES path_2002;
127
128 resultset_2002_return(");
129
130 resultset_2002_return("MATCH (a0) WHERE (a0.id = 1);
131 resultset_2002_return("START a0;
132 resultset_2002_return("STOP a0;
133 resultset_2002_return("RETURN a0;");
134
135 IF (path_2002_1144747) {
136 resultset_2002_return("FROM table_diamond");
137 }
138 resultset_2002_return(
139 "MATCH (a0) WHERE (a0.id = 1);
140 resultset_2002_return(" + path_2002);
141);
142
143 }
144
145 // Sort result
146 resultset_2002_sort_by_order_of = [
147 path_2002_2002_resultset_2002;
148 resultset_2002_return(");
149];
150
151 SELECT path_2002_2002_resultset_2002;
152 SELECT path_2002_2002_resultset_2002;
153 SELECT path_2002_2002_resultset_2002;
154 SELECT path_2002_2002_resultset_2002;
155 }
156
157 // Sort result
158 resultset_2002_sort_by_order_of = [
159 path_2002_2002_resultset_2002;
160 resultset_2002_return(");
161];
162
163 SELECT path_2002_2002_resultset_2002;
164 SELECT path_2002_2002_resultset_2002;
165 SELECT path_2002_2002_resultset_2002;
166 SELECT path_2002_2002_resultset_2002;
167 }
168
169 // Sort result
170 resultset_2002_sort_by_order_of = [
171 path_2002_2002_resultset_2002;
172 resultset_2002_return(");
173];
174
175 SELECT path_2002_2002_resultset_2002;
176 SELECT path_2002_2002_resultset_2002;
177 SELECT path_2002_2002_resultset_2002;
178 SELECT path_2002_2002_resultset_2002;
179 }
180
181 // Sort result
182 resultset_2002_sort_by_order_of = [
183 path_2002_2002_resultset_2002;
184 resultset_2002_return(");
185];
186
187 SELECT path_2002_2002_resultset_2002;
188 SELECT path_2002_2002_resultset_2002;
189 SELECT path_2002_2002_resultset_2002;
190 SELECT path_2002_2002_resultset_2002;
191 }
192
193 // Sort result
194 resultset_2002_sort_by_order_of = [
195 path_2002_2002_resultset_2002;
196 resultset_2002_return(");
197];
198
199 SELECT path_2002_2002_resultset_2002;
200 SELECT path_2002_2002_resultset_2002;
201 SELECT path_2002_2002_resultset_2002;
202 SELECT path_2002_2002_resultset_2002;
203 }
204
205 // Sort result
206 resultset_2002_sort_by_order_of = [
207 path_2002_2002_resultset_2002;
208 resultset_2002_return(");
209];
210
211 SELECT path_2002_2002_resultset_2002;
212 SELECT path_2002_2002_resultset_2002;
213 SELECT path_2002_2002_resultset_2002;
214 SELECT path_2002_2002_resultset_2002;
215 }
216
217 // Sort result
218 resultset_2002_sort_by_order_of = [
219 path_2002_2002_resultset_2002;
220 resultset_2002_return(");
221];
222
223 SELECT path_2002_2002_resultset_2002;
224 SELECT path_2002_2002_resultset_2002;
225 SELECT path_2002_2002_resultset_2002;
226 SELECT path_2002_2002_resultset_2002;
227 }
228
229 // Sort result
230 resultset_2002_sort_by_order_of = [
231 path_2002_2002_resultset_2002;
232 resultset_2002_return(");
233];
234
235 SELECT path_2002_2002_resultset_2002;
236 SELECT path_2002_2002_resultset_2002;
237 SELECT path_2002_2002_resultset_2002;
238 SELECT path_2002_2002_resultset_2002;
239 }
240
241 // Sort result
242 resultset_2002_sort_by_order_of = [
243 path_2002_2002_resultset_2002;
244 resultset_2002_return(");
245];
246
247 SELECT path_2002_2002_resultset_2002;
248 SELECT path_2002_2002_resultset_2002;
249 SELECT path_2002_2002_resultset_2002;
250 SELECT path_2002_2002_resultset_2002;
251 }
252
253 // Sort result
254 resultset_2002_sort_by_order_of = [
255 path_2002_2002_resultset_2002;
256 resultset_2002_return(");
257];
258
259 SELECT path_2002_2002_resultset_2002;
260 SELECT path_2002_2002_resultset_2002;
261 SELECT path_2002_2002_resultset_2002;
262 SELECT path_2002_2002_resultset_2002;
263 }
264
265 // Sort result
266 resultset_2002_sort_by_order_of = [
267 path_2002_2002_resultset_2002;
268 resultset_2002_return(");
269];
270
271 SELECT path_2002_2002_resultset_2002;
272 SELECT path_2002_2002_resultset_2002;
273 SELECT path_2002_2002_resultset_2002;
274 SELECT path_2002_2002_resultset_2002;
275 }
276
277 // Sort result
278 resultset_2002_sort_by_order_of = [
279 path_2002_2002_resultset_2002;
280 resultset_2002_return(");
281];
282
283 SELECT path_2002_2002_resultset_2002;
284 SELECT path_2002_2002_resultset_2002;
285 SELECT path_2002_2002_resultset_2002;
286 SELECT path_2002_2002_resultset_2002;
287 }
288
289 // Sort result
290 resultset_2002_sort_by_order_of = [
291 path_2002_2002_resultset_2002;
292 resultset_2002_return(");
293];
294
295 SELECT path_2002_2002_resultset_2002;
296 SELECT path_2002_2002_resultset_2002;
297 SELECT path_2002_2002_resultset_2002;
298 SELECT path_2002_2002_resultset_2002;
299 }
300
301 // Sort result
302 resultset_2002_sort_by_order_of = [
303 path_2002_2002_resultset_2002;
304 resultset_2002_return(");
305];
306
307 SELECT path_2002_2002_resultset_2002;
308 SELECT path_2002_2002_resultset_2002;
309 SELECT path_2002_2002_resultset_2002;
310 SELECT path_2002_2002_resultset_2002;
311 }
312
313 // Sort result
314 resultset_2002_sort_by_order_of = [
315 path_2002_2002_resultset_2002;
316 resultset_2002_return(");
317];
318
319 SELECT path_2002_2002_resultset_2002;
320 SELECT path_2002_2002_resultset_2002;
321 SELECT path_2002_2002_resultset_2002;
322 SELECT path_2002_2002_resultset_2002;
323 }
324
325 // Sort result
326 resultset_2002_sort_by_order_of = [
327 path_2002_2002_resultset_2002;
328 resultset_2002_return(");
329];
330
331 SELECT path_2002_2002_resultset_2002;
332 SELECT path_2002_2002_resultset_2002;
333 SELECT path_2002_2002_resultset_2002;
334 SELECT path_2002_2002_resultset_2002;
335 }
336
337 // Sort result
338 resultset_2002_sort_by_order_of = [
339 path_2002_2002_resultset_2002;
340 resultset_2002_return(");
341];
342
343 SELECT path_2002_2002_resultset_2002;
344 SELECT path_2002_2002_resultset_2002;
345 SELECT path_2002_2002_resultset_2002;
346 SELECT path_2002_2002_resultset_2002;
347 }
348
349 // Sort result
350 resultset_2002_sort_by_order_of = [
351 path_2002_2002_resultset_2002;
352 resultset_2002_return(");
353];
354
355 SELECT path_2002_2002_resultset_2002;
356 SELECT path_2002_2002_resultset_2002;
357 SELECT path_2002_2002_resultset_2002;
358 SELECT path_2002_2002_resultset_2002;
359 }
360
361 // Sort result
362 resultset_2002_sort_by_order_of = [
363 path_2002_2002_resultset_2002;
364 resultset_2002_return(");
365];
366
367 SELECT path_2002_2002_resultset_2002;
368 SELECT path_2002_2002_resultset_2002;
369 SELECT path_2002_2002_resultset_2002;
370 SELECT path_2002_2002_resultset_2002;
371 }
372
373 // Sort result
374 resultset_2002_sort_by_order_of = [
375 path_2002_2002_resultset_2002;
376 resultset_2002_return(");
377];
378
379 SELECT path_2002_2002_resultset_2002;
380 SELECT path_2002_2002_resultset_2002;
381 SELECT path_2002_2002_resultset_2002;
382 SELECT path_2002_2002_resultset_2002;
383 }
384
385 // Sort result
386 resultset_2002_sort_by_order_of = [
387 path_2002_2002_resultset_2002;
388 resultset_2002_return(");
389];
390
391 SELECT path_2002_2002_resultset_2002;
392 SELECT path_2002_2002_resultset_2002;
393 SELECT path_2002_2002_resultset_2002;
394 SELECT path_2002_2002_resultset_20
```

- 

Berdasarkan praktikum yang telah dilakukan, program berhasil mengimplementasikan struktur data *Binary Tree* dan *Graph Traversal* menggunakan bahasa *Java*. Pada implementasi *Binary Tree*, program dapat membentuk pohon *biner*, menghitung jumlah simpul, melakukan pencarian

data, serta menampilkan hasil *traversal Inorder*, *Preorder*, dan *Postorder*. Selain itu, struktur pohon juga dapat ditampilkan dalam bentuk visual sederhana pada *console*.

Pada implementasi *Graph Traversal*, program berhasil merepresentasikan graf menggunakan *Adjacency List* serta menerapkan algoritma *Breadth First Search (BFS)* dan *Depth First Search (DFS)*. Hasil penelusuran dapat ditampilkan melalui antarmuka *GUI* sehingga pengguna dapat memilih titik awal dan tujuan, kemudian melihat jalur yang ditemukan oleh masing-masing algoritma.

### **3. Kesimpulan**

Berdasarkan hasil praktikum, dapat disimpulkan bahwa struktur data *Binary Tree* digunakan untuk menyimpan data dalam bentuk hierarki dan dapat ditelusuri menggunakan metode *Inorder*, *Preorder*, dan *Postorder*. Sementara itu, struktur data *Graph* dapat digunakan untuk merepresentasikan hubungan antar simpul dan ditelusuri menggunakan algoritma *BFS* maupun *DFS* untuk menemukan jalur dari suatu titik ke titik lainnya.